

LangGraph Agent Architecture

Understanding document for the Main HR Orchestrator and role-specific agents

Purpose

This backend uses LangGraph as the orchestration framework. A single Main HR Orchestrator creates an agent run, invokes a compiled StateGraph, and returns a stable API response. The graph uses one LLM Router Agent plus deterministic fallback routing to send requests into specialized role paths.

Implemented shape

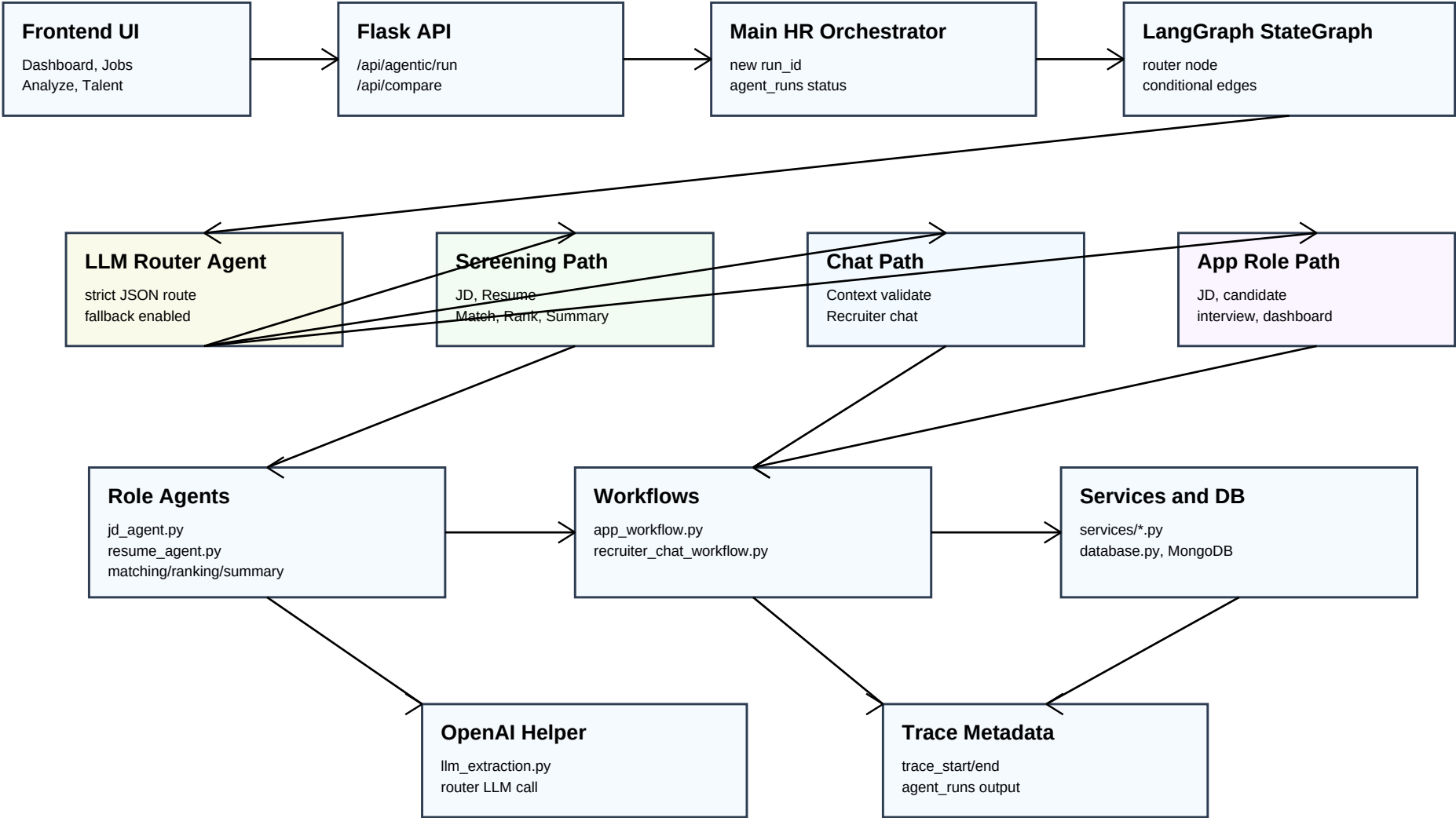
- API endpoint: routes/agent_routes.py receives JSON or multipart requests.
- Main orchestrator: orchestration/main_orchestrator.py creates run_id and records running/completed/failed state.
- LangGraph graph: orchestration/hr_graph.py builds StateGraph nodes and conditional edges.
- Router: agents/router_agent.py returns one route from screening, chat, jd, candidate, interview, dashboard, reports, unsupported.
- Screening path: JD Agent -> Resume Agent -> Matching Agent -> Ranking Agent -> Summary Agent -> Final Response.
- Application paths: JD, candidate, interview, dashboard, reports use app_role_agent -> run_app_workflow.
- Chat path: recruiter_context -> recruiter_chat_agent -> Final Response.

Important concept

LangGraph is used for stateful orchestration and routing. LangChain is present as a dependency, but the implemented graph nodes call local Python agents, workflows, services, database helpers, and the existing OpenAI helper instead of LangChain runnable chains.

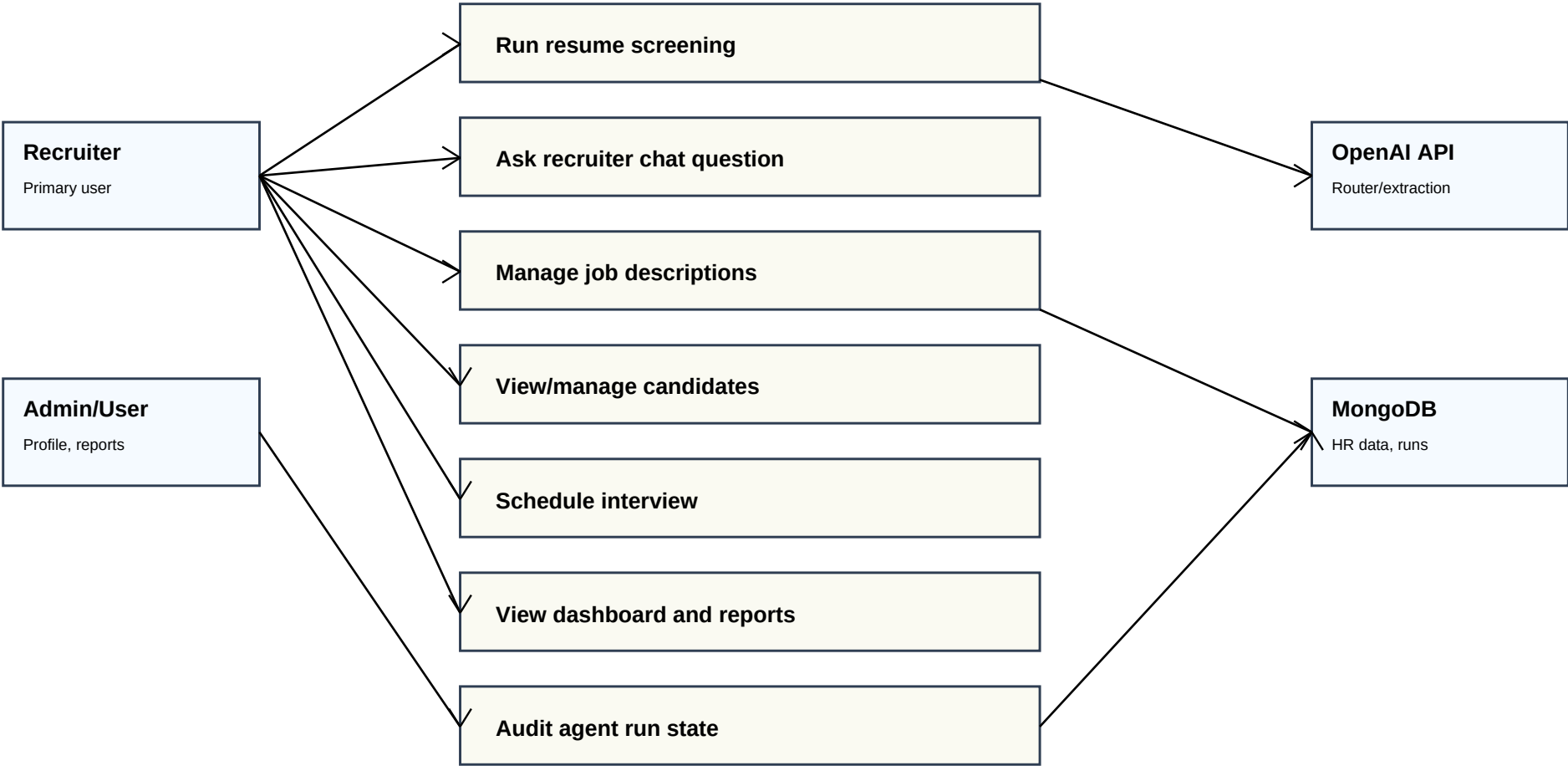
Architecture Diagram

Runtime components and boundaries



Use Case Diagram

Actors and LangGraph-backed capabilities

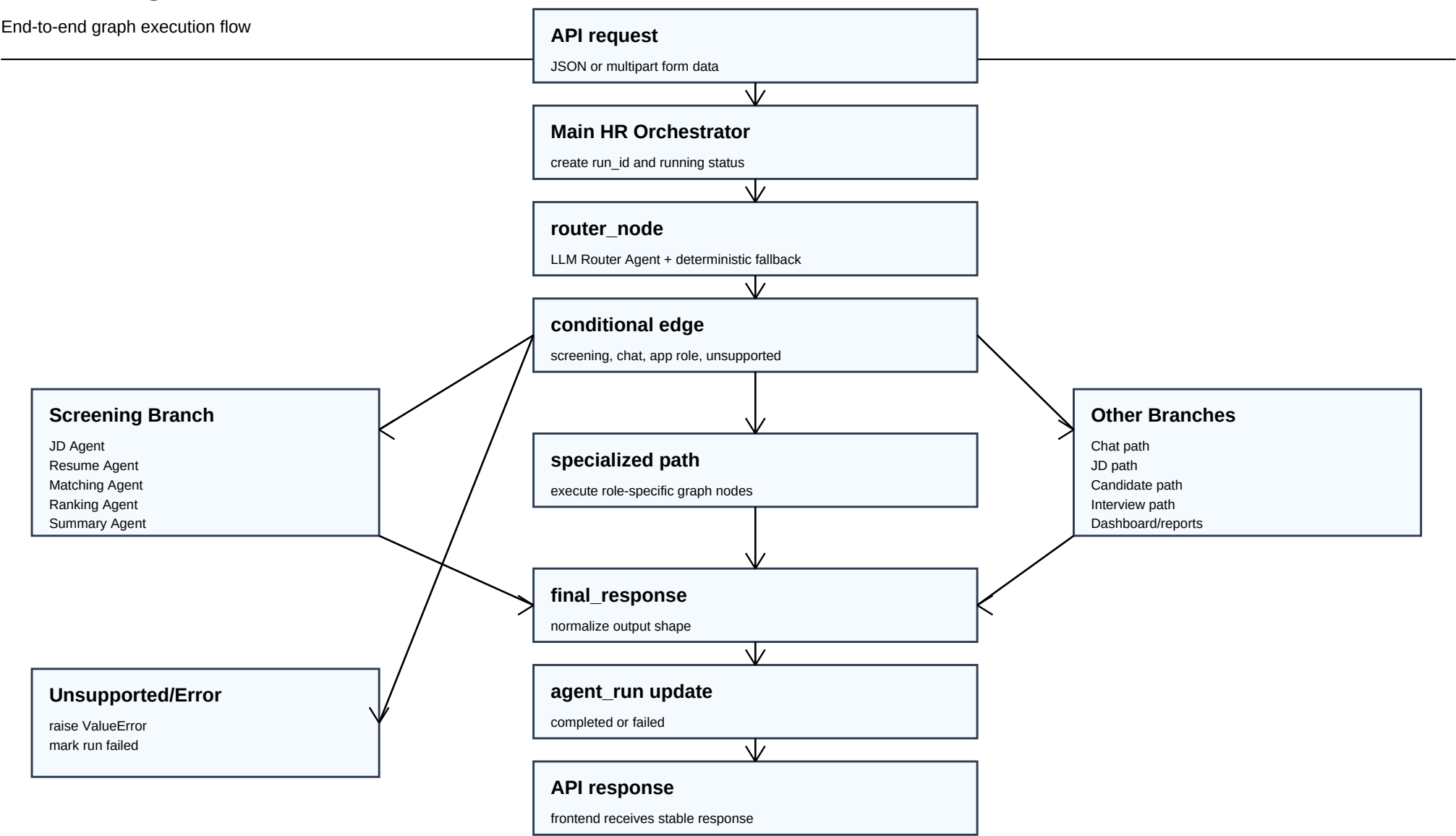


Use case boundary: users still call the same Flask APIs; LangGraph is an internal orchestration layer.

The Router Agent is the only component that decides the global route.

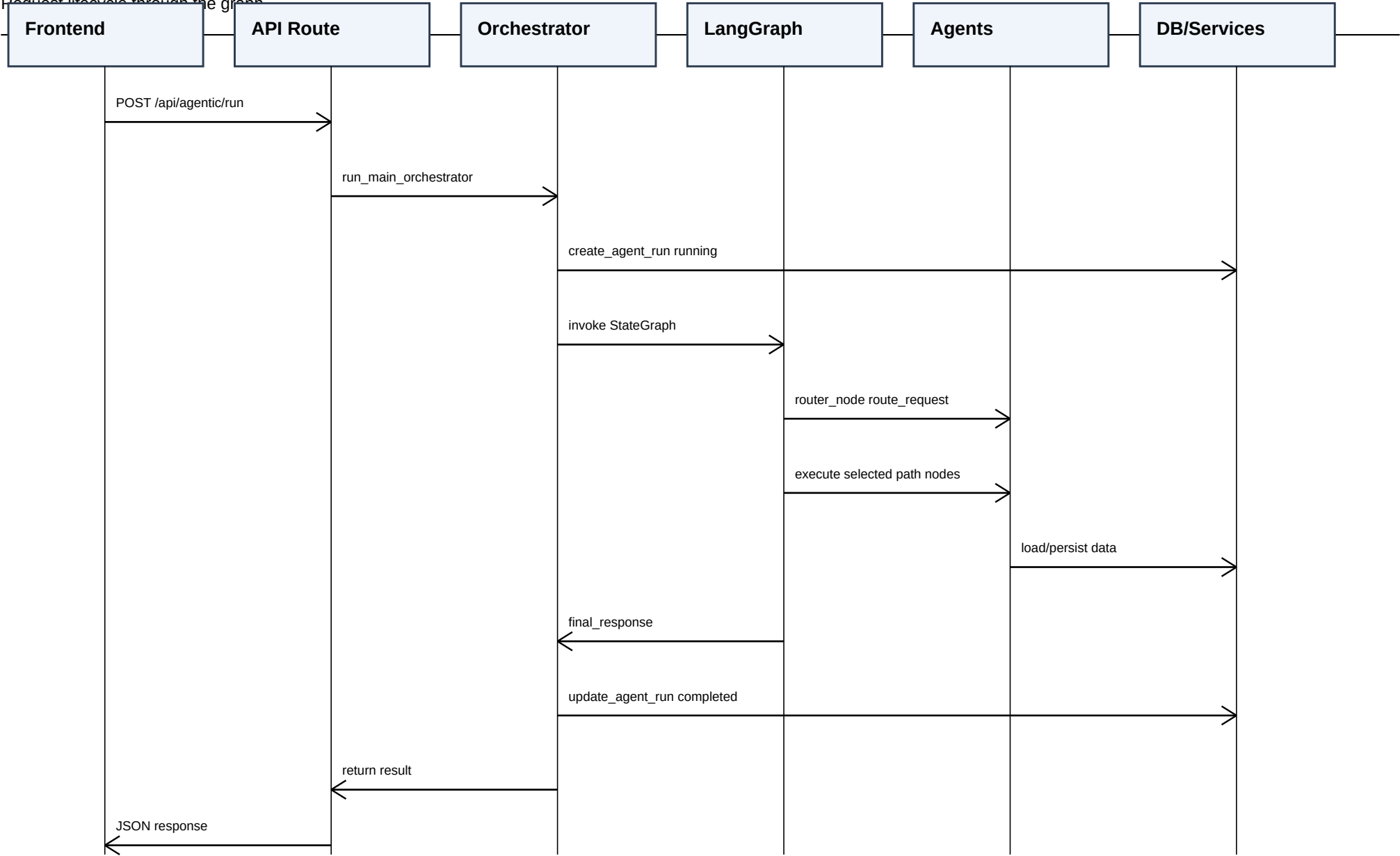
Flow Diagram

End-to-end graph execution flow



UML Sequence Diagram

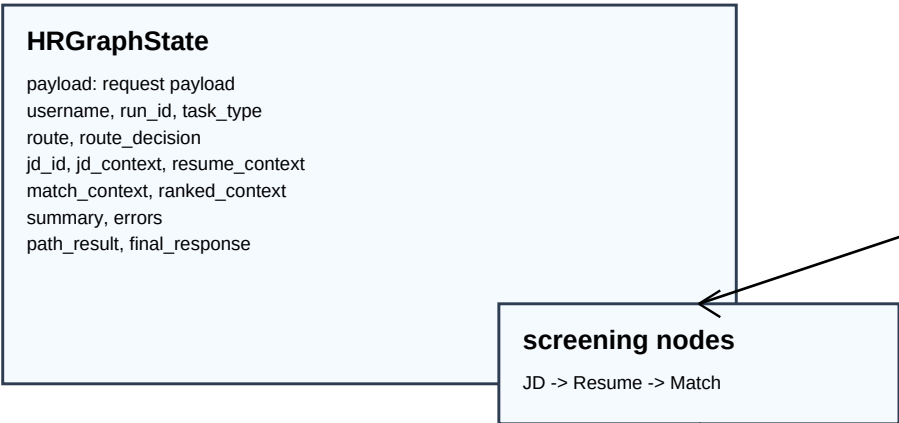
Request lifecycle through the graph



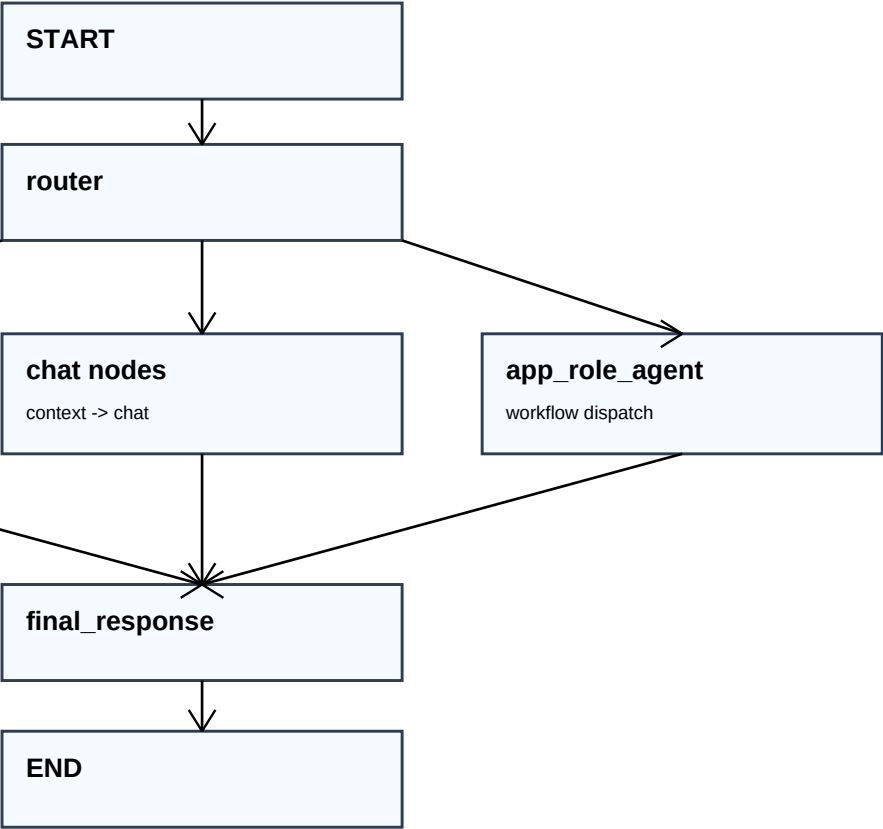
UML State and Class View

Shared state object and graph node responsibilities

StateGraph state model



State transitions



Node rule

- Each graph node receives HRGraphState and returns a partial state update.
- Only router_node chooses the global route.
- final_response_node creates the API-facing response shape.
- Fatal exceptions are caught by the main orchestrator and mark the run failed.